

Hunter College
Department of Computer Science
CSCI 39548: Practical Web Development
Summer 2026
Assignment 3 — Reading Tracker (React + Tailwind)
Instructor: *Sourya Saha* (sourya.saha24@login.cuny.edu)

Weight	10% of course grade
Released	Monday, June 15, 2026
Due	Monday, June 29, 2026, by 6:40 PM (before class)
Submission	Brightspace — paste a link to your public GitHub repository + a link to your merged pull request
Type	Individual

1 Overview

Build a **personal reading tracker** as a single-page web app using **React** (with Vite + TypeScript) and **Tailwind CSS**. The app lets a user search for books from a public API, add them to their library, move them through reading stages (*To Read* → *Reading* → *Finished*), filter and sort their list, see reading stats, and have their library survive a page refresh.

This assignment is the React counterpart to A2: A2 wired the DOM by hand with `querySelector` and a manual `render()` function; A3 hands that job off to React. The **patterns are the same** — state, render, events, fetch — but the implementation is now declarative.

Build it as we teach it. The assignment is released the same day we start React so you can grow your solution alongside the lectures. A suggested rhythm:

- **After the first React class** — Scaffold the Vite project, get a `<BookCard />` component rendering some hardcoded props, and lay out the page shell.
- **After the second React class** — Wire up state with `useState`, render the library list with keys, build the add/edit forms, and add the `useEffect` + `fetch` call against the Open Library search API.
- **After React + Tailwind** — Split into clean sub-components, add Tailwind styling, make it responsive, add a dark-mode toggle.
- **By Mon Jun 29** — Polish, validate, handle empty/error/loading states, push the final PR.

*Note: React + Tailwind only. Use Vite as the build tool and TypeScript for your source files (.tsx). Style with Tailwind utility classes, not hand-written CSS files. Do **not** use Bootstrap, Material UI, Chakra, or any other component library. Routing libraries (React Router etc.) are also not needed here.*

2 Learning Objectives

- **CO5** — Compose a React UI from small, reusable function components that share data through props.
- **CO6** — Manage local UI state with `useState` (and where it fits, `useReducer`); express side effects with `useEffect`; fetch and render data from a public HTTP API.
- **CO7** — Style a responsive interface using Tailwind utility classes and a class-based dark-mode strategy.

3 Part 1 — Core React App

Build a Vite + React + TypeScript project. The app must support **all** of the following:

- **Search books** via a search bar that calls the **Open Library** search API:
<https://openlibrary.org/search.json?q=<query>&limit=10>
Show the matching books (title, author, first publish year, cover thumbnail if available) in a results panel.
- **Add a book** from the search results into the user's library with a default status of **"To Read"**. The same book added twice should be detected and not duplicated.
- **Status pipeline** — each book in the library has a status of **to-read** / **reading** / **finished**. The user can move a book forward or back through the pipeline from the card.
- **Delete** a book from the library.
- **Rate** a finished book on a 1–5 star scale (only available when a book is in the **finished** state).
- **Filter** the library view by status (**All**, **To Read**, **Reading**, **Finished**).
- **Sort** the library by **title**, **author**, or **date added**, ascending or descending — controlled by the user.
- **Stats bar** — show counts (**X to read** · **Y reading** · **Z finished**) and the average rating of finished books. Both update reactively on every change.
- **Empty state** — when the library has no books (or the filter matches none), show a friendly empty-state message instead of a blank panel.

3.1 React Requirements

- Project scaffolded with **Vite**:

```
npm create vite@latest -- --template react-ts.
```
- State is owned at the **right level** — lift it as high as it needs to go and no higher.
- At minimum, **four distinct components** in their own files (e.g. `App.tsx`, `BookCard.tsx`, `SearchBar.tsx`, `StatsBar.tsx`).

- **Lists are rendered from arrays** using `.map()`, each item with a stable, unique **key** (do not use the array index unless the list never reorders).
- Use `useState` for local state. If your `App.tsx` ends up with a tangle of 6+ `useState` calls all about the library, `useReducer` is the right choice — use it.
- At least one `useEffect` — for the book-search fetch, the `localStorage` persistence, or both.
- **No direct DOM access** (no `document.querySelector`, no `innerHTML`). All UI updates flow through React state.

4 Part 2 — Tailwind Styling

- Tailwind installed and wired up (either the Vite plugin or PostCSS — your choice; we use the Vite plugin in class).
- The app is styled **with Tailwind utility classes** in JSX — not with hand-written CSS files.
- **Responsive** — the layout must work cleanly on a phone-width screen ($\leq 400\text{px}$ wide) and on a desktop screen ($\geq 1024\text{px}$ wide). The book grid should reflow.
- **Dark mode** — a toggle button switches between light and dark themes. The user's preference must be **remembered across page reloads** (`localStorage`). Use Tailwind's class-based dark mode (`dark:` variants), not media queries.
- Visual polish — consistent spacing, readable typography, meaningful hover/focus states on buttons and links. The app should not look like a default unstyled HTML page.

5 Part 3 — Persistence + Lifecycle

- On every meaningful state change (library updated, theme toggled), save to `localStorage`.
- On page load, **read state back from `localStorage`** — the user's library and theme preference survive a refresh.

- If `localStorage` is empty or its contents can't be parsed, fall back to a sane default (empty library, light theme) — don't crash.
- The book-search `fetch` must show:
 - A **loading state** while the request is in flight (spinner, skeleton, or “Searching...” text), and
 - A **clear error message** on network failure or bad response — do **not** silently swallow the error.

6 Submission

Submit on **Brightspace**. Paste **two links**:

1. Your public GitHub repository URL — e.g.
`https://github.com/yourusername/reading-tracker`
2. Your merged pull request URL — e.g.
`https://github.com/yourusername/reading-tracker/pull/1`

The grader will **clone the repo, run `npm install` and `npm run dev`**, and open it in the browser. Make sure those two commands are enough.

*Note: Verify your links work in a private/incognito window before submitting. Make sure **`node_modules/`** is **not** committed.*

7 Git/GitHub Requirements

Same workflow as A1 and A2 — this is not new:

- Public GitHub repository with a `README.md` and a `.gitignore` (include `node_modules/` and `dist/`).
- Work on a **feature branch**, not directly on `main`.
- **Several meaningful commits** with clear messages.
- **At least one pull request** merged into `main`.

A clean workflow is part of the rubric. A working app committed straight to `main` in one giant commit called “done” will lose points.

8 Grading Rubric (100 pts)

Criterion	Pts
Core features work — search, add, status pipeline, delete, rate, filter, sort, stats all correct	30
Component decomposition — sensible split, props typed, state lifted to the right level	10
Hooks used correctly — <code>useState</code> / <code>useEffect</code> (and <code>useReducer</code> if used) idiomatic, no anti-patterns	10
Lists & keys — stable unique keys, list reorder/filter doesn't break the UI	5
fetch + loading + error — search request with loading state, error path handled visibly	10
Tailwind styling — utility classes used in JSX, no hand-written CSS files for layout	5
Responsive + dark mode — works on phone + desktop widths; dark-mode toggle persists	10
localStorage persistence — library + theme survive refresh; handles missing/corrupt data	10
Git workflow — feature branch, PR, merge, clear commits, README, <code>.gitignore</code>	5
Code readability — naming, structure, no dead code, no <code>console.log</code> spam, no <code>any</code> types	5

9 Notes

- **Late policy:** see the syllabus. Up to one week late (with prior contact) at up to -20% per day.
- **Use of AI:** allowed and encouraged, but you must be able to **explain every line you submit**. Don't ship React you can't read.

- **TypeScript, not plain JavaScript.** The Vite template generates `.tsx` files — keep them that way. Type your props, your state, and the shape of API responses. Use of `any` will cost readability points.
- **The Open Library API requires no key.** Test the search URL in the browser address bar first to see the JSON shape. Cover image URL pattern:
`https://covers.openlibrary.org/b/id/{cover_i}-M.jpg`
(the field `cover_i` comes back in the search result; some books don't have one — handle that gracefully).
- **DevTools = your friend.** The React DevTools extension (Chrome / Firefox) lets you inspect component state and props live — install it early.
- **Stuck?** Discuss among yourselves first, then consult web resources (React docs, MDN, Stack Overflow). If you're still stuck, bring questions to class or email to set up office hours.
- **What this prepares you for:** A4 adds routing (multiple pages), global state (Zustand), and remote data caching (TanStack Query). The final project replaces `localStorage` with a real backend (Express + Prisma + Postgres). The component patterns you build here are exactly what you'll keep using.

End of Assignment 3